

Introducción a la Programación I

Clase 2: Strings e Input

¿Qué es un String?

Un **string** es una secuencia de caracteres.

En Python se puede escribir con comillas simples o dobles — el resultado es el mismo:

```
nombre = 'Ada Lovelace'  
nombre = "Ada Lovelace"
```

Podés usar comillas dobles dentro de un string con comillas simples, y viceversa:

```
mensaje = "Le dijo 'hola' al pasar."
```

Concatenación de Strings

Con el operador `+` podés unir (concatenar) strings:

```
nombre = "Ada"  
apellido = "Lovelace"  
nombre_completo = nombre + " " + apellido  
print(nombre_completo)
```

Ada Lovelace

Longitud de un String: `len()`

La función `len()` devuelve la cantidad de caracteres de un string:

```
nombre = "Python"  
print(len(nombre))
```

6

Incluye espacios y caracteres especiales:

```
print(len("hola mundo")) # 10
```

Métodos de String: `title()`, `upper()`, `lower()`

Los **métodos** son funciones asociadas a un objeto. Se llaman con un punto.

```
nombre = "ada loveLace"

print(nombre.title())    # Ada Lovelace
print(nombre.upper())    # ADA LOVELACE
print(nombre.lower())    # ada loveLace
```

`lower()` es muy útil para comparar strings sin importar mayúsculas.

Comparar Strings

Podés usar operadores de comparación con strings:

Operador	Significado
<code>==</code>	igual a
<code>!=</code>	distinto de
<code><</code>	menor que
<code>></code>	mayor que
<code><=</code>	menor o igual
<code>>=</code>	mayor o igual

```
print("gato" == "gato")    # True
print("gato" != "perro")   # True
print("abc" < "abd")       # True (orden alfabético)
```

Eliminar Espacios en Blanco

Los espacios al principio o al final de un string suelen ser un problema.

```
lenguaje = "  python  "

print(lenguaje.rstrip())    # "  python"
print(lenguaje.lstrip())    # "python  "
print(lenguaje.strip())     # "python"
```

Muy útil al procesar input del usuario.

f-strings: Formatear Strings con Variables

Los **f-strings** permiten insertar variables directamente dentro de un string:

```
nombre = "Ada"  
apellido = "Lovelace"  
  
mensaje = f"Hola, {nombre} {apellido}!"  
print(mensaje)
```

Hola, Ada Lovelace!

También podés poner expresiones:

```
print(f"2 + 2 = {2 + 2}") # 2 + 2 = 4
```


Caracteres Especiales: `\n` y `\t`

`\n` agrega un salto de línea, `\t` agrega una **tabulación**:

```
print("Lenguajes:\n\tPython\n\tJavaScript\n\tRuby")
```

```
Lenguajes:  
    Python  
    JavaScript  
    Ruby
```

Buscar en un String: `find()`

`find(sub)` devuelve el **índice más bajo** donde se encuentra el substring.

Si no lo encuentra, devuelve `-1`.

```
mensaje = "hola mundo"

print(mensaje.find("mundo")) # 5
print(mensaje.find("xyz"))  # -1
```

Los índices empiezan en `0`.

Strings Multilínea

Con **triple comillas** (`"""` o `'''`) podés crear strings que abarcan varias líneas:

```
poema = """Había una vez  
un programador  
que amaba Python."""  
  
print(poema)
```

```
Había una vez  
un programador  
que amaba Python.
```

Verificar si un Substring Está Presente

Usá `in` y `not in` para buscar substrings:

```
frase = "me gusta Python"

print("Python" in frase)      # True
print("JavaScript" in frase)  # False
print("Java" not in frase)    # True
```

Slicing: Acceder a Caracteres Individuales

Podés acceder a un carácter por su **índice** (empieza en `0`):

```
lenguaje = "Python"

print(lenguaje[0])    # P
print(lenguaje[1])    # y
print(lenguaje[-1])   # n   (último carácter)
print(lenguaje[-2])   # o   (anteúltimo)
```

Los **índices negativos** cuentan desde el final.

Slicing: Rango de Caracteres

Podés extraer una porción de un string con `[start:end]` :

- El índice de inicio está **incluido**
- El índice de fin está **excluido**

```
lenguaje = "Python"

print(lenguaje[0:3])    # Pyt
print(lenguaje[2:5])    # tho
```

Slicing: Omitir Índices

Si omitís el inicio, arranca desde el principio.

Si omitís el fin, llega hasta el final.

```
lenguaje = "Python"

print(lenguaje[:3])    # Pyt    (desde el inicio hasta índice 2)
print(lenguaje[3:])    # hon    (desde índice 3 hasta el final)
print(lenguaje[:])     # Python (copia completa)
```

Slicing: Saltar Caracteres y Orden Inverso

Con `[start:end:step]` podés saltar de a `step` caracteres:

```
texto = "abcdefgh"

print(texto[::2])    # aceg  (de a dos)
print(texto[1::2])   # bdfh
```

Para invertir un string usá `[::-1]` :

```
print("Python"[::-1])  # nohtyP
```


Los Strings son Inmutables

No podés modificar un string cambiando uno de sus caracteres directamente:

```
lenguaje = "Python"  
lenguaje[0] = "J"    # ✗ TypeError!
```

Para "modificar" un string hay que crear uno nuevo:

```
lenguaje = "J" + lenguaje[1:]  
print(lenguaje)    # Jython
```

Métodos: `replace()` y `count()`

`replace(old, new)` — reemplaza todas las apariciones de un substring:

```
frase = "me gusta Java, Java es genial"  
nueva = frase.replace("Java", "Python")  
print(nueva)    # me gusta Python, Python es genial
```

`count(substring)` — cuenta cuántas veces aparece un substring:

```
print(frase.count("Java"))    # 2
```

Input del Usuario: `input()`

La función `input()` muestra un mensaje y espera que el usuario escriba algo.
Siempre devuelve un **string**.

```
nombre = input("¿Cómo te llamas? ")  
print(f"Hola, {nombre}!")
```

```
¿Cómo te llamas? Ada  
Hola, Ada!
```

Parsear Input como Número

Como `input()` devuelve un string, hay que convertirlo para hacer operaciones matemáticas:

```
edad_str = input("¿Cuántos años tenés? ")  
edad = int(edad_str)  
print(f"El año que viene tenés {edad + 1} años.")
```

Para números decimales usá `float()` :

```
altura = float(input("¿Cuánto medís (en metros)? "))
```

Combinando `input()` con Métodos de String

```
nombre = input("Ingresa tu nombre: ").strip().title()
print(f"Bienvenido/a, {nombre}!")
```

Se pueden encadenar métodos: primero se ejecuta `strip()`, después `title()`.

Documentación de Python

Python tiene documentación oficial con todos los métodos de string disponibles:

 docs.python.org/3/library/stdtypes.html#string-methods

Algunos métodos útiles además de los vistos:

Método	Qué hace
<code>startswith(s)</code>	True si empieza con <code>s</code>
<code>endswith(s)</code>	True si termina con <code>s</code>
<code>isdigit()</code>	True si todos los caracteres son dígitos
<code>split(sep)</code>	Divide el string en una lista

Resumen de la Clase

Concepto	Ejemplo
Concatenación	<code>"hola" + " " + "mundo"</code>
<code>len()</code>	<code>len("Python")</code> → <code>6</code>
Métodos	<code>"ada".title()</code> → <code>"Ada"</code>
f-strings	<code>f"Hola {nombre}"</code>
Slicing	<code>"Python"[0:3]</code> → <code>"Pyt"</code>
<code>in</code> / <code>not in</code>	<code>"py" in "python"</code> → <code>True</code>
<code>input()</code>	<code>nombre = input("Nombre: ")</code>
<code>int()</code> / <code>float()</code>	<code>edad = int(input("Edad: "))</code>